ReST meets ReAct: Self-Improvement for Multi-Step Reasoning LLM Agent

Renat Aksitov[†1], Sobhan Miryoosefi[†1], Zonglin Li[†1], Daliang Li[†1], Sheila Babayan[†2], Kavya Kopparapu[†2], Zachary Fisher[1], Ruiqi Guo[1], Sushant Prakash[1], Pranesh Srinivasan[3], Manzil Zaheer[2], Felix Yu[1], and Sanjiv Kumar[1]

[1]Google Research, [2]Google DeepMind, [3]Google
[†]Core contributors

arXiv:2312.10003v1 [cs.CL] 15 Dec 2023

Abstract

Answering complex natural language questions often necessitates multi-step reasoning and integrating external information. Several systems have combined knowledge retrieval with a large language model (LLM) to answer such questions. These systems, however, suffer from various failure cases, and we cannot directly train them end-to-end to fix such failures, as interaction with external knowledge is non-differentiable. To address these deficiencies, we define a ReAct-style LLM agent with the ability to reason and act upon external knowledge. We further refine the agent through a ReST-like method that iteratively trains on previous trajectories, employing growing-batch reinforcement learning with AI feedback for continuous self-improvement and self-distillation. Starting from a prompted large model and after just two iterations of the algorithm, we can produce a fine-tuned small model that achieves comparable performance on challenging compositional question-answering benchmarks with two orders of magnitude fewer parameters.

1 Introduction

For many simple natural language tasks, like basic question-answering or summarization, we can relatively easily decide whether the final output is good or bad, collect large amounts of such data, and train the language models using these outcomes as feedback. At the same time, for more complex problems, outcome-based systems are often insufficient, and a process supervision approach has recently gained much attention as a more promising alternative (Reppert et al. (2023)). There is explosive growth in techniques (Gao et al. (2023); Madaan et al. (2023)), frameworks (Dohan et al. (2022); Khattab et al. (2023b)), and libraries (Liu (2022), Chase (2022)) for defining process-based workflows with LLMs through human-understandable task decompositions. Many such decompositions involve interaction with external tools / APIs / environments, in which case the corresponding multi-step workflow is generally referred to as an LLM agent (Xi et al. (2023)), a system capable of performing a sequence of actions to achieve a goal.

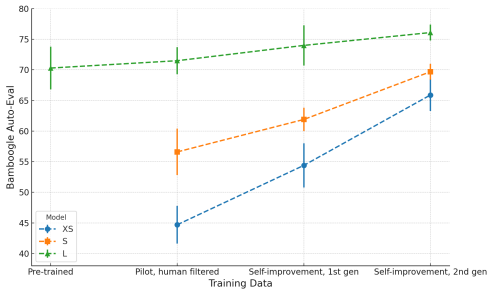

Figure 1: *Agent self-improvement and self-distillation. Bamboogle auto-eval, mean accuracy and standard deviation over 10 runs, (%)*

Let's consider the task of answering complex, open-ended questions, where the agent needs to use a search API to look up multiple pieces of information before composing a paragraph-length answer. One popular approach for building such agents with LLMs is the ReAct method (Yao et al., 2022), which involves interleaving chain-of-thought reasoning with actions and observations during sev-

eral thought-action-observation rounds. In this work, we follow the general ReAct format for our Search Agent while designing the corresponding few-shot prompts to produce long-form, explicitly attributable final answers (*cf.* Nakano et al. (2021)).

It is natural to ask next how to deal with failure cases of such an agent and how to improve its performance and robustness. For outcome-based systems, the solution is usually straightforward: we just collect more human-labeled data. However, acquiring such data is much more challenging and expensive for process-based systems: a significantly larger amount of data is needed (Uesato et al. (2022); Lightman et al. (2023)), and it is generally harder for humans to determine an optimal multi-step trajectory.

To address the lack of and difficulty in obtaining multi-step human-labeled data, we focus on improving the quality of the agent with self-critique, AI feedback, and synthetic data generation. Specifically for the latter, we adapt the recently proposed (Gulcehre et al. (2023)) Reinforced Self-Training (`ReST`) algorithm towards agentic setups. The inner-outer loop flow of `ReST` remains the same: in the outer loop ("grow"), the dataset is grown by sampling from the latest policy, and in the inner loop ("improve"), the policy is improved on a fixed dataset via ranking or filtering with reward model. In our case, sampling during "grow" means producing a multi-step trajectory to completion, and ranking as part of "improve" is done directly with LLM call rather than with a distilled reward model of human preferences.

We measure the overall performance of the Search Agent by its ability to answer diverse compositional questions that were manually verified to be unanswerable directly by a search engine (Bamboogle dataset (Press et al., 2023) and a sequel dataset that we constructed ourselves, BamTwoogle). While both datasets are small, they have enough statistical power to capture the effects we are interested in studying. For example, Figure 1 provides a high-level overview of the agent performance on Bamboogle for different model sizes and showcases the boost with more iterations of `ReST` (all our synthetic data is produced with the large model, so for smaller models, it is the performance of the distillation).

To summarize, our contributions are the following:

- We build a flavor of ReAct agent with self-critique for the task of long-form question answering.
- We define a proxy evaluation metric for the agent based on Bamboogle and BamTwoogle datasets, with a strong emphasis on auto-eval.
- We demonstrate that the performance of the agent could be effectively improved through Rest-style iterative fine-tuning on its reasoning traces.
- Furthermore, we do it purely from stepwise AI feedback without using human-labeled training data.
- Finally, we show that the synthetic data produced as part of this iterative process could be used for distilling the agent into one or two orders of magnitude smaller models with performance comparable to the pre-trained teacher agent.

2 BACKGROUND: SEARCH AGENT

This section describes Search Agent, a flavor of ReAct (Yao et al., 2022) agent with Reflexion (Shinn et al., 2023). It uses web search as a tool to generate long-form, explicitly attributable answers for diverse knowledge-seeking open-ended questions. The agent's flow proceeds as follows (Figure 2):

1. Agent receives a question and starts executing a search loop:
    - Agent **decides** whether it needs additional information to answer the question.
    - If "yes", it calls the search tool, **summarizes** the received snippets, and goes back to the decision step.
    - If "no", it terminates the search loop.
2. Based on the information collected as part of the search loop, the agent generates the first attempt (draft) of the **answer**.
3. It then performs two additional self-revision calls before producing the final answer:

- One to verify that the answer is **relevant** to the original question,
- And another to check that the answer is **grounded** in the retrieved snippets.

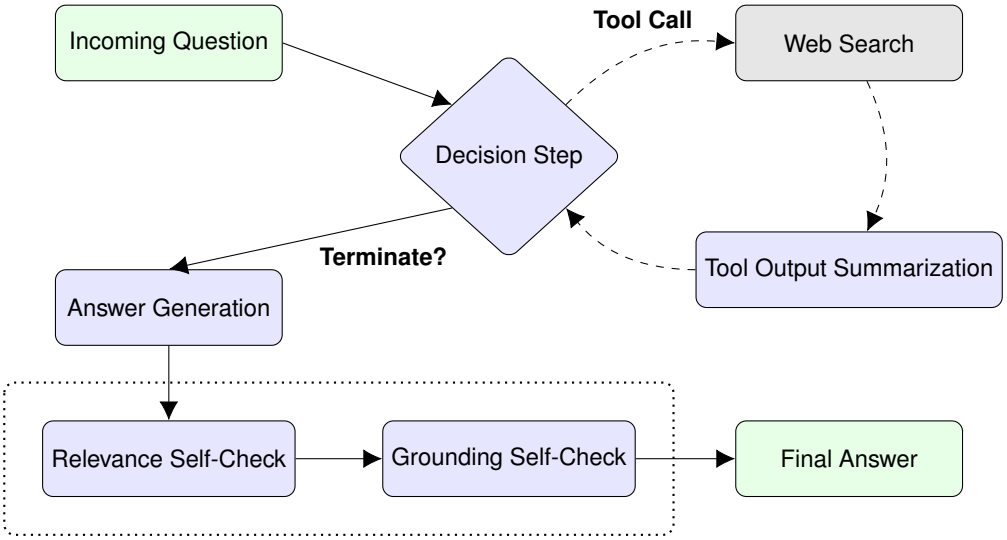

Figure 2: *A state machine of the Search Agent flow. Each blue shape corresponds to a single LLM call and defines a separate type of the reasoning step.*

3 METHODS

3.1 PROMPTING

We first define the prompted flow for the Search Agent by manually constructing few-shot prompts for each of the five reasoning steps from Figure 2. Notably, we choose to format our prompts as Python code (see Appendix for the prompts of different steps, Listings 1 - 6).

The decision to use the "code as the prompt" approach is motivated by the following observations:

- There is often a need to parse the LLM's output for integration with other systems and tools, which is much easier to do if the model's input and output are well-structured.
- At the same time, code uniquely combines a structured aspect (keywords and syntax) with a natural language aspect (comments and descriptive naming).
- Moreover, LLMs are capable of both reading and writing code.

To summarize, since code is naturally structured and easy to parse, it could serve as an excellent medium for communication with the model. Accordingly, within the "code as the prompt" paradigm, LLM is expected to understand the code in the input and continue it as valid Python in the output, which turns out to be challenging tasks for the smaller models. Out of the (pre-trained) models we've tried, only the `PaLM 2-L` (Anil et al., 2023) can perform it consistently well. Hence, this is the model that we use to produce reasoning trajectories from few-shot prompts.

3.2 IMPLEMENTATION DETAILS

To run Search Agent, we use `PaLM 2` "base" models of different sizes (`XS`, `S` and `L`), both pre-trained and fine-tuned. We usually produce multiple samples (with $T = 0.5$, see Section 4.2 for details on temperature selection) for each step and then select the one with the lowest perplexity (4 samples for `L` and `S`, 16 for `XS`) to continue the trajectory forward. As a search tool, we use internal Google Q&A API that returns top-k snippets, and we typically request top-3 snippets per query from it. We further limit the number of total searches to at most 10 per single agent trajectory (via the REMAINING_SEARCHES field in the prompts). Search Agent preserves the trajectory state by storing all the actions taken so far in the PAST_ACTIONS field.

3.3 Input Data

We use the following four datasets to provide initial questions for Search Agent trajectories:

- **HotpotQA** (Yang et al., 2018), a multi-hop reasoning QA dataset, where the system has to reason with information taken from more than one document to arrive at the answer.
- **Eli5** (Fan et al., 2019), a dataset for long-form question answering (LFQA), a task that requires elaborate and in-depth answers to open-ended questions. The dataset was built from the Reddit forum "Explain Like I'm Five" (ELI5), r/explainlikeimfive.
- **Eli5-askH** (Blagojevic, 2022), similar to above, but built from the Reddit forum r/askhistorians, a subreddit where users may ask questions or start discussions about history.
- **Eli5-askS** (Blagojevic, 2022), as above, but from a subreddit r/askscience (*"ask a science question, get a science answer"*).

We randomly selected 500 questions from the training splits of each dataset and ended up with 2000 diverse, challenging questions in total. We don't use any other information from these datasets, like labels (e.g., we don't do any filtering by matching the correct answer from HotpotQA, and we don't use these datasets' validation splits for hyperparameters tuning or performance assessment).

3.4 Fine-Tuning

We simply split each completed Search Agent trajectory into the reasoning steps and build a fine-tuning mixture with those steps. We use full fine-tuning for all the experiments. Given that fine-tuning costs increase sharply for larger models, we do as many experiments as possible with `XS` model.

3.5 Ranking "Reward" Model

As mentioned previously, we produce multiple samples for each reasoning step in the agent's trajectory and typically choose the sample that minimizes perplexity to continue the trajectory forward or to build a fine-tuning mixture. At the same time, we might be able to do better than that by utilizing a more sophisticated way of selecting the best sample. To this effect, we employ an instruction-tuned `PaLM 2-L` and prompt it with the model input, multiple sampled outputs, and guidance on how to rank them (the prompt is available in the Appendix, Listing 8). We can then use the highest-ranked sample for fine-tuning instead of the default sample chosen based on the perplexity value.

In this part, our approach differs from `ReST`, which uses threshold-based filtering with a reward model (RM) trained on human preference data. What we do is closer to RAFT (Dong et al., 2023), where the reward model ranks sampled responses to select high-scoring subsets for model fine-tuning, and the RM rankings matter much more than absolute scores. Notably, we mainly do the LLM-based selection off-policy, i.e., by updating the current action used for fine-tuning. The on-policy trajectory rollouts use perplexity.

3.6 Iterative Self-Improvement

Now we have all the pieces for the self-improvement algorithm:

- Start with a model capable of performing Search Agent task at a certain level, for example, with prompted `PaLM 2-L` model. Collect reasoning trajectories from this model based on our set of 2000 initial questions (essentially the **"grow"** stage of `ReST`, with the difference that we keep the set of initial questions fixed).
- Convert the trajectories into the fine-tuning mixture. Apply re-ranking with RM during the conversion (this is roughly equivalent to the **"improve"** stage of `ReST`, though we only do one iteration of "improve").
- Fine-tune the new model (of the same size) on this mixture and verify that it's performing better than the original model (we will discuss *how* to do it in the following section). Repeat the process, starting with this new, better model.

Finally, we can also train *smaller* models on the fine-tuning data from the different iterations of self-improvement, which will naturally give us a self-distillation algorithm.

4 EVALUATION

4.1 BAMBOOGLE

Our primary evaluation vehicle is the Bamboogle dataset (Press et al., 2023). It is a semi-adversarial dataset of 2-hop questions (**125** in total) that were selected to be unanswerable by direct Google search, but where both required pieces of evidence could be found in Wikipedia. When the Search Agent's performance on Bamboogle is improving, we can assume that it generally becomes better in using search as a tool.

Given the open-ended nature of the answers generated by the Search Agent (Figure 3), we can't reliably use the exact match as a metric. Instead, we either manually judge correctness or run auto-eval with a separate call to the `PaLM 2-L` "base" model.

4.2 AUTO-EVAL

While strict human evaluations are preferable, they are time-consuming. They also don't scale well (doing one human eval is much easier than doing five), which leads to a high variance of such evals in our case: the agent's trajectories are stochastic (as a reminder, we use non-zero temperature when sampling reasoning steps), but we can't easily reduce the variance by increasing the number of repetitions per question with human evals.

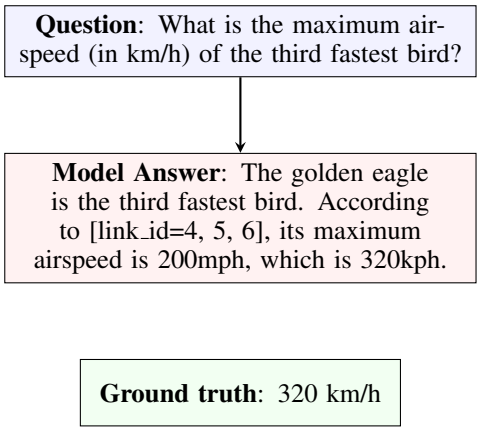

Figure 3: *A Bamboogle question with an example of the long form answer by the model*

We solve both of these problems by introducing LLM-based auto-eval (the full auto-eval prompt is available in the Appendix, Listing 7) and by verifying its alignment with human ratings for the Bamboogle trajectories on which we previously conducted human evals. We compared the auto-eval with human evaluations on a diverse set of agents and found it highly correlated with human evaluation scores. Specifically, the Pearson correlation is 0.98 with $p = 6.6 \times 10^{-8}$ and Spearman correlation is 0.83 with $p = 0.0015$. Given that it is much cheaper to run auto-eval, we can now use a large number of repetitions to reduce variance. We typically aggregate auto-eval over ten repetitions (i.e., by producing ten different trajectories for each Bamboogle question).

First and foremost, we use Bamboogle auto-eval to estimate the final model performance but also to answer various questions that one would typically use a validation set for:

- What is the optimal sampling temperature for the agent? ($T = 0.5$)
- Which checkpoints should we choose for different model sizes? (step 9K for `XS`, 5K for `S`, 3.5K for `L`)
- Should we proceed with another iteration of self-improvement?
- What is the performance impact of using several trajectories per question on the fine-tuned model?
- Should we use self-checks? Are they helping or hurting the results? (helping slightly, see Figure 4)

Importantly, we never use Bamboogle as a training set, as we neither tune our prompts on Bamboogle nor use questions from it to generate fine-tuning trajectories.

4.3 BAMTWOOGLE

Given the small size of Bamboogle and our use of it as an analog of the validation set for Search Agent, there is a danger of overfitting. To guard against such a possibility, we introduce a new dataset, BamTwoogle, to serve as a test set. We use BamTwoogle exclusively to measure the final performance of the models.

The BamTwoogle dataset was written to be a complementary, slightly more challenging sequel to Bamboogle. It also addresses some of the shortcomings of Bamboogle we discovered while performing human evals. Specifically, we ensured that all the questions required 2+ steps to answer. Due to changes in the search algorithm, this is no longer the case with all Bamboogle questions, and it is possible to solve some of them with a single search.

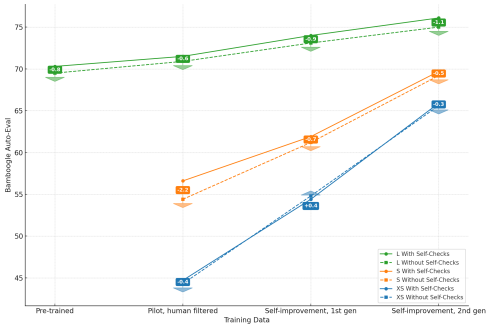

Figure 4: *Agent self-improvement and self-distillation, with and without self-critique steps. Bamboogle auto-eval, mean accuracy and standard deviation over ten runs, (%)*

Like Bamboogle, BamTwoogle is a small (**100** questions in total), handcrafted collection of information-seeking questions. The topics and question formats vary, but in general, BamTwoogle adheres to the following guidelines

Questions

- The majority of questions require two searches or reasoning steps (like Bamboogle), but some of them need 3 or 4
- Must have been manually checked to ensure the answer doesn't appear on the first page of Google search results

Expected answers

- Should not be ambiguous
- Should not be prone to change over time, either due to the phrasing of the question or to the nature of the answer
- Should account for multiple versions of proper names, etc., where appropriate
- Should prefer Wikipedia as the source of truth for facts (preference given to topics/articles not flagged for incompleteness, lack of sources, etc.)

5 EXPERIMENTS

5.1 PILOT

Alongside the main self-improvement setup, described in Section 3, we also generate a simpler "pilot" data with 500 trajectories, where the initial questions are selected from HotpotQA and Eli5 datasets only (i.e., smaller and without Eli5-askH or Eli5-askS). We use the default (based on a min perplexity, no RM re-ranking) best actions for building the pilot's fine-tuning mixture. Furthermore, we manually

Table 1: *Agent self-improvement and self-distillation, Bamboogle auto-eval, mean accuracy and standard deviation over 10 runs, (%)*

Training Data	XS	S	L
Pre-trained	N/A	N/A	$70.3^{\pm 3.5}$
Pilot, human filtered	$44.7^{\pm 3.1}$	$56.6^{\pm 3.8}$	$71.5^{\pm 2.2}$
Self-improvement, 1st gen	$54.4^{\pm 3.6}$	$61.9^{\pm 1.9}$	$74.0^{\pm 3.3}$
Self-improvement, 2nd gen	$65.9^{\pm 2.6}$	$69.7^{\pm 1.3}$	$76.1^{\pm 1.3}$

review the fine-tuning data and filter
out about 30% of the examples that are "bad" in some way: an unhelpful query, empty thoughts, summary missing important information, etc. This pilot data serves as a fine-tuning baseline.

5.2 SELF-IMPROVEMENT AND SELF-DISTILLATION

The main results are presented in Figure 1 and Table 1. As discussed previously, each cell in the table shows a mean (with the corresponding std) of auto-eval over 10 Bamboogle runs for the particular model.

We start with a pre-trained (prompted) `PaLM 2-L` model and use it to generate both 500 pilot trajectories and (independently) 2000 trajectories for 1st iteration ("1st gen") of self-improvement. We then fine-tune `PaLM 2-L`, `PaLM 2-S`, and `PaLM 2-XS` models on the resulting mixtures.

Table 2: *Human evals, Bamboogle and BamTwoogle, acc (%)*

	Pre-trained (`L`)	2nd gen (`XS`)	2nd gen (`S`)	2nd gen (`L`)
Bamboogle	68.8	67.2	68.0	74.4
BamTwoogle	68.0	63.0	63.0	74.0

Next, we use the `PaLM 2-L` model fine-tuned on 1st gen data to generate trajectories for the 2nd iteration ("2nd gen") of self-improvement. We build 2nd gen data with 8000 trajectories (using the same 2000 initial questions, each repeated four times; analogous to a "grow" stage in `ReST`). As previously, we fine-tuned each of the three models on the new 2nd gen mixture.

As a final verification, we also do human evals over a single Bamboogle and BamTwoogle run for each of the 2nd gen models (Table 2), as well as the original pre-trained `PaLM 2-L` model for comparison.

5.3 ABLATIONS

What is the effect of human filtering? Surprisingly, we have found that fine-tuning on filtered data results in a small performance drop (2.5%) versus unfiltered pilot mixture (Table 3, pilot columns). We hypothesize that this happens due to a combination of 2 factors:

- The reduced size of the filtered mixture makes it harder for the model to learn the proper format of the prompts,
- Our filtering only affects the immediate "bad" example, not the whole trajectory; the "bad" step would often be preserved in the other fine-tuning examples as part of the PAST_ACTIONS field.

Should we use multiple trajectories per question? Turns out, it helps to use two trajectories per question instead of 1 (2.2% gain) in the fine-tuning mixture, but more than that doesn't improve performance significantly (Table 3, 2nd gen columns).

Table 3: *Impact of the quality and size of the training data for fine-tuned `PaLM 2-XS` models*

	Pilot, human filtered	Pilot, unfiltered	1st gen	2nd gen (1x)	2nd gen (2x)	2nd gen (4x)
Total trajectories	500	500	2000	2000x1	2000x2	2000x4
Training examples	3015	4518	17970	18007	36238	72424
Bamboogle auto-eval	$44.7^{\pm 3.1}$	$47.2^{\pm 3.1}$	$54.4^{\pm 3.6}$	$63.4^{\pm 1.7}$	$65.6^{\pm 1.8}$	$65.9^{\pm 2.6}$

More data vs better data. From the same Table 3, we can conclude that the quality of the data (e.g., 9% gain, when going from 1st gen to 2nd gen (1x) while keeping the size of the data roughly the same) matters more than its quantity. Notably, better data also reduces the variance of evaluation trajectories.

Effect of self-critique. The multi-step setup of the agent allows us to easily measure the effect of the self-critique steps on the overall agent performance. To do this, we simply take all the Bamboogle trajectories used for Table 1 and run auto-eval on them, but over the "Answer Generation" step, rather than over "Final Answer", as we normally do (Figure 2). As seen from Figure 4, the self-critique results in a small but measurable positive boost (on the order of 0.5-1.0% for most models). The detailed numbers are provided in Table 4 in the Appendix.

6 Discussion

Process Supervision. It's important to emphasize again that we don't use labels from the training data as a signal during trajectory collection. This is possible due to combining a process-based approach (i.e., defining agent as a state machine) with high-temperature exploration, AI feedback (zero-shot "reward" model used for actions re-ranking), and state-wise fine-tuning over completed trajectories. In other words, in this setup, the model can learn something useful even from the states that eventually lead to the wrong final answer. More importantly, it learns to handle open-ended questions that don't have a single well-defined correct answer in the first place, akin to many questions from Eli5.

Auto-Eval. Some of the properties listed above, like non-greedy sampling and long-form answer generations in particular, bring with them additional challenges in terms of agent evaluation. We both need to measure the quality of the long-form final answer for a specific trajectory and account for stochasticity between different agent trajectories for the same input. This, on the one hand, elevates the value of having robust auto-eval aligned with human raters and, on the other, increases the computational costs significantly due to the need to run agent trajectories multiple times, as well as the use of `PaLM 2-L` model for auto-eval.

Self-Critique. Despite those computational costs, multiple benefits come from having reliable auto-eval in terms of measuring the impact of various agent's hyperparameters. As an example, let's consider the self-critique steps. With the aid of auto-eval, we were able to establish that they have a small but positive effect on the overall performance of our multi-step reasoning setup. This contrasts the recent observations (Huang et al., 2023) that adding self-critique hurts the performance in an outcome-based CoT setup. Simultaneously, we can also notice that a positive effect of self-critique depends on the model size (larger for larger models) but does not seem to be affected by the self-improvement process.

Limitations and Future Directions. While this work lays a foundation for reasoning agents' self-improvement, it has numerous limitations due to computational and time constraints: manually constructed prompts, small evals, a limited set of models, and only a single tool, to name just a few.

Future work could explore if the same self-improvement algorithm applies to multiple tool settings and, especially, if the ability to handle unseen tools could be improved in such a way. If the latter is more similar to self-critique and doesn't improve under `ReST`-like iterative training, what changes are required to enable self-improvement for both?

Another open question is a saturation point. How many additional iterations of self-improvement can we undertake past the 2nd one that still results in non-trivial gains? What does the saturation look like for smaller models? Will they all eventually converge to the same performance, or will the smaller models always be capped by the performance of the initial prompted large model?

7 Related Work

Following WebGPT (Nakano et al., 2021), we are tackling the task of long-form question answering (Krishna et al., 2021), in which the language agent uses web search as a tool to generate final answers with explicit references for the retrieved passages. While WebGPT is focused on imitation learning and RL from a large number of human demonstrations, our work aims to minimize human involvement. The only labeled demonstrations we use as part of the training are few-shot exemplars in the prompts for the agent's reasoning steps (see Appendix, A.1).

Setting up language agents with manually designed few-shot prompts is the most common practice (Press et al. (2023); Yao et al. (2022); Shinn et al. (2023)), but there are some exceptions. For example, DSP (Khattab et al., 2023a) tunes the few-shot demonstrations for the prompts automatically, leveraging some amount of labeled training examples for optimization purposes, and can further fine-tune specific components of the agent.

Unlike prompting, agent's fine-tuning is done less often (Nakano et al. (2021); Yao et al. (2022); Chen et al. (2023)). The closest to our fine-tuning setup is probably FireAct (Chen et al., 2023), with the main difference being that we don't use human labels for training or data filtering. Instead, we are building synthetic data with self-improvement from AI feedback.

Some relevant papers for self-improvement include STAR (Zelikman et al., 2022), ReST (Gulcehre et al., 2023), ReST^{EM} (Singh et al., 2023), and RAFT (Dong et al., 2023). Unlike STAR and ReST^{EM}, we don't use the correctness of the answer as a signal. And, unlike ReST and RAFT, we don't have the proper reward model trained on human preferences. Moreover, all 4 of these papers target outcome-based systems, while we focus on a process-based one.

8 Conclusion

This work demonstrates that the ReST-like approach with AI feedback could be effectively applied to a multi-step reasoning LLM agent. We show that it is a relatively simple and efficient way to iteratively build high-quality synthetic data for agent self-improvement. Moreover, this increasingly higher quality data could simultaneously be used for distilling a multi-step agent into several magnitudes smaller models while preserving most of the performance from the large teacher model.

References

Rohan Anil, Andrew M Dai, Orhan Firat, Melvin Johnson, Dmitry Lepikhin, Alexandre Passos, Siamak Shakeri, Emanuel Taropa, Paige Bailey, Zhifeng Chen, et al. Palm 2 technical report. *arXiv preprint arXiv:2305.10403*, 2023.

Vladimir Blagojevic. Long-form qa beyond eli5: an updated dataset and approach, 2022. URL towardsdatascience.com/long-form-qa-beyond-eli5-an-updated-dataset-and-approach-319cb841aabb.

Harrison Chase. Langchain. https://github.com/hwchase17/langchain, 2022.

Baian Chen, Chang Shu, Ehsan Shareghi, Nigel Collier, Karthik Narasimhan, and Shunyu Yao. Fireact: Toward language agent fine-tuning, 2023.

David Dohan, Winnie Xu, Aitor Lewkowycz, Jacob Austin, David Bieber, Raphael Gontijo Lopes, Yuhuai Wu, Henryk Michalewski, Rif A. Saurous, Jascha Sohl-dickstein, Kevin Murphy, and Charles Sutton. Language model cascades, 2022.

Hanze Dong, Wei Xiong, Deepanshu Goyal, Yihan Zhang, Winnie Chow, Rui Pan, Shizhe Diao, Jipeng Zhang, Kashun Shum, and Tong Zhang. Raft: Reward ranked finetuning for generative foundation model alignment, 2023.

Angela Fan, Yacine Jernite, Ethan Perez, David Grangier, Jason Weston, and Michael Auli. ELI5: long form question answering. *CoRR*, abs/1907.09190, 2019. URL http://arxiv.org/abs/1907.09190.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Pal: Program-aided language models, 2023.

Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Ksenia Konyushkova, Lotte Weerts, Abhishek Sharma, Aditya Siddhant, Alex Ahern, Miaosen Wang, Chenjie Gu, et al. Reinforced self-training (rest) for language modeling. *arXiv preprint arXiv:2308.08998*, 2023.

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet, 2023.

Omar Khattab, Keshav Santhanam, Xiang Lisa Li, David Hall, Percy Liang, Christopher Potts, and Matei Zaharia. Demonstrate-search-predict: Composing retrieval and language models for knowledge-intensive nlp, 2023a.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. Dspy: Compiling declarative language model calls into self-improving pipelines, 2023b.

Kalpesh Krishna, Aurko Roy, and Mohit Iyyer. Hurdles to progress in long-form question answering, 2021.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step, 2023.

Jerry Liu. Llamaindex. `https://github.com/jerryjliu/llama_index`, 2022.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback, 2023.

Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. Webgpt: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*, 2021.

Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A. Smith, and Mike Lewis. Measuring and narrowing the compositionality gap in language models, 2023.

Justin Reppert, Ben Rachbach, Charlie George, Luke Stebbing, Jungwon Byun, Maggie Appleton, and Andreas Stuhlmüller. Iterated decomposition: Improving science q&a by supervising reasoning processes, 2023.

Noah Shinn, Beck Labash, and Ashwin Gopinath. Reflexion: an autonomous agent with dynamic memory and self-reflection. *arXiv preprint arXiv:2303.11366*, 2023.

Avi Singh, John D. Co-Reyes, Rishabh Agarwal, Ankesh Anand, Piyush Patil, Peter J. Liu, James Harrison, Jaehoon Lee, Kelvin Xu, Aaron Parisi, Abhishek Kumar, Alex Alemi, Alex Rizkowsky, Azade Nova, Ben Adlam, Bernd Bohnet, Gamaleldin Elsayed, Hanie Sedghi, Igor Mordatch, Isabelle Simpson, Izzeddin Gur, Jasper Snoek, Jeffrey Pennington, Jiri Hron, Kathleen Kenealy, Kevin Swersky, Kshiteej Mahajan, Laura Culp, Lechao Xiao, Maxwell L. Bileschi, Noah Constant, Roman Novak, Rosanne Liu, Tris Warkentin, Yundi Qian, Ethan Dyer, Behnam Neyshabur, Jascha Sohl-Dickstein, and Noah Fiedel. Beyond human data: Scaling self-training for problem-solving with language models, 2023.

Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. Solving math word problems with process- and outcome-based feedback, 2022.

Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, Rui Zheng, Xiaoran Fan, Xiao Wang, Limao Xiong, Yuhao Zhou, Weiran Wang, Changhao Jiang, Yicheng Zou, Xiangyang Liu, Zhangyue Yin, Shihan Dou, Rongxiang Weng, Wensen Cheng, Qi Zhang, Wenjuan Qin, Yongyan Zheng, Xipeng Qiu, Xuanjing Huang, and Tao Gui. The rise and potential of large language model based agents: A survey, 2023.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. *CoRR*, abs/1809.09600, 2018. URL `http://arxiv.org/abs/1809.09600`.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. Star: Bootstrapping reasoning with reasoning, 2022.

A APPENDIX

A.1 PROMPTS

Reasoning steps. All our reasoning prompts are n-shot (with $n > 1$), though we only show fragments, limited to the first exemplar:

- The **decision** step prompt is shown in Listing 1 (the full prompt is 9-shot).
- The **summarization** prompt is presented in Listing 2 (the full prompt is 6-shot).
- The **answer generation** prompt is 5-shot, Listing 3 shows a 1-shot fragment.
- The prompt for **relevance self-check** is presented in Listing 5 (the full prompt is 6-shot).
- Finally, the prompt for **grounding self-check** is shown in Listing 6 (the full prompt is 5-shot).
- Both self-checks use the common prefix from Listing 4.

Auto-eval. The full 5-shot prompt for auto-eval is presented in Listing 7.

Ranking "Reward" Model. The full prompt of the reward model is shown in Listing 8. Notice that this is the only prompt that is 0-shot and doesn't use "code as the prompt" approach, as it's intended to be used with an instruction-tuned model.

A.2 ADDITIONAL DATA

Table 4: *Agent self-improvement and self-distillation, Bamboogle auto-eval, mean accuracy and standard deviation over 10 runs, (%). Evaluation before self-critique steps*

	XS	S	L
Pre-trained	N/A	N/A	$69.5^{\pm 2.8}$ (-0.8)
Pilot, human filtered	$44.3^{\pm 3.0}$ (-0.4)	$54.4^{\pm 4.1}$ (-2.2)	$70.9^{\pm 3.0}$ (-0.6)
Self-improvement, 1st gen	$54.8^{\pm 3.7}$ (+0.4)	$61.2^{\pm 2.5}$ (-0.7)	$73.1^{\pm 3.0}$ (-0.9)
Self-improvement, 2nd gen	$65.6^{\pm 3.0}$ (-0.3)	$69.2^{\pm 1.8}$ (-0.5)	$75.0^{\pm 1.3}$ (-1.1)

Listing 1: *Decision step prompt (fragment)*

```
"""Implement an agent capable of answering complex queries by potentially search multiple
     times.
"""
import dataclasses


class Action:
  """Base class for different actions."""
  ...


@dataclasses.dataclass
class ActionWrapper:
  """Encapsulates the reasoning as well as the selected action.

  Attributes:
    thoughts: Record your thoughts on why we should do this action.
    action: The actually selected action.
  """
  thoughts: str
  action: Action

@dataclasses.dataclass
class Search(Action):
  """The Google search command.

  Attributes:
    query: The query to be sent to Google.
  """
  query: str


@dataclasses.dataclass
class Terminate(Action):
  """Command to terminate the search sequence."""
  ...


##########################
# Example 1:
##########################

ORIGINAL_QUESTION: str = 'I want to buy the Ikea Klippan loveseat, but I\'m not sure whether
     it can fit into my 2019 Honda Odyssey. Could you check whether I need to fold down the
     seat?'
PAST_ACTIONS: List[Action] = [
Search(query='ikea klippan loveseat dimension',
thoughts="To answer the ORIGINAL_QUESTION, we need to find the size of the Ikea Klippan
     loveseat and the cargo size of the 2019 Honda Odyssey, and then compare them. Let's start
      by finding the size of the Ikea Klippan loveseat.",
),
SelectLink(selected_links=[
  ResultItem(link_id=1, link_text='KLIPPAN Loveseat, Vissle gray - IKEA',
             snippet='<lh> Measurements </lh> <ul> <li> Width: 70 7/8 &quot; </li> <li> Depth:
                   34 5/8 &quot; </li> <li> Height: 26 &quot; </li> <li> Height under
                  furniture: 4 3/8 &quot; </li> <li> Seat depth: 21 1/4 &quot; </li> <li> Seat
                   height: 16 7/8 &quot; </li> </ul>'),],
grounded_summarization='According to [link_id=1] the dimension of the ikea klippan loveseat is
      Width: 70 7/8"; Depth: 34 5/8"; Height: 26".',
thoughts="We still need to find the cargo size of the 2019 Honda Odyssey to answer the
     ORIGINAL_QUESTION.",
),
]
REMAINING_SEARCHES: int = 4
ORIGINAL_QUESTION: str = 'I want to buy the Ikea Klippan loveseat, but I\'m not sure whether
     it can fit into my 2019 Honda Odyssey. Could you check whether I need to fold down the
     seat?'

ACTION_SELECTED = ActionWrapper(thoughts="The past result gives us the dimension of the love
     seat. We indeed need to find the cargo size of the 2019 Honda Odyssey.", action=Search(
     query='2019 Honda Odyssey cargo size'))  # [END]

##########################
# Example 2:
##########################
```

Listing 2: *Summarization prompt (fragment)*

```
"""Implementing step 2 of Search and Answer procedure: Search result filtering and
     summarization.
"""

##########################
# Example 1:
##########################

ORIGINAL_QUESTION: str = "What's the efficiency of the solar panels on the garden solar lights
     ?"
PAST_ACTIONS: List[Action] = [
Search(query='garden solar light panel efficiency',
thoughts="Let's just rephrase the search query a bit. The intention of the original question
     is pretty clear.",
),
]
CURRENT_SEARCH_RESULTS = SearchResult(links=[
  ResultItem(link_id=17, link_text='Use Solar Outdoor Lighting for Energy Efficiency - Sepco
       Solar',
             snippet='Depending on the type of lighting you install and the wattage of the
                  fixtures, this can eat up a budget quickly. Instead, going with a solar
                  outdoor lighting system to provide the required light can greatly reduce the
                   project costs, help improve the environment, and keep electrical costs in
                  the bank where it belongs.'),
  ResultItem(link_id=18, link_text='How Do Solar Garden Lights Work? - The OutdoorLights Store
       ',
             snippet='The solar panel converts solar energy into electricity<br><br> The
                  energy absorbed by the photovoltaic cells in the panel creates electrical
                  charges. These charges move in response to an electrical field in the solar
                  panel&#39;s cells, which ultimately causes electricity to flow.'),
  ResultItem(link_id=19, link_text='10 things you should know about the garden solar lights',
             snippet='The solar panels are generally made from two types of cells,
                  monocrystalline or polycrystalline. Both of them are low cost, however,
                  different panel technology can lead to different efficiency under different
                  conditions.'),
])
ORIGINAL_QUESTION: str = "What's the efficiency of the solar panels on the garden solar lights
     ?"

# [link_id=17] is an article about garden solar light, which doesn't have anything specific
     about the solar panels. Not selected.
# [link_id=18] is an article on how to best install those solar lights. Not relevant to the
     solar panels used. Not selected.
ACTION_SELECTED: LinkSelection = LinkSelection(grounded_summarization='According to [link_id
     =19], there are two types of solar cells used for garden solar lights, the
     monocrystalline or polycrystalline panels.', thoughts="It seems like non of the links
     shows the efficiency number. However, link 19 provides some hints on what to search next.
      We can look into the efficiency number of those two types of panels.", selected_link_ids
     =[19])  # [END]

##########################
# Example 2:
##########################
```

Listing 3: *Answer generation prompt (fragment)*

```
"""Implementing step 3 of the multi-round search agent: Answer generation based on all the
     collected search results.
"""

##########################
# Example 1:
##########################

ORIGINAL_QUESTION: str = "What's the white powdery residual often found on a humidifier?"
PAST_ACTIONS: List[Action] = [
Search(query='what is humidifier',
thoughts="Let's solve this step by step. The person is asking about the residual found in
     humidifier. Let's first define what does humidifier do.",
),
SelectLink(selected_links=[
  ResultItem(link_id=1, link_text='Humidifiers: What They Do for Health, Uses, Types, and More
       ',
             snippet='What is a humidifier? Humidifiers are devices that add moisture to the
                  air to prevent dryness that can cause irritation in many parts of the body.
                  Humidifiers can be particularly effective for treating dryness of the skin,
                  nose, throat, and lips. They can also ease some symptoms caused by the flu
                  or common cold.'),
  ResultItem(link_id=3, link_text='5 humidifier uses: Benefits and risks - Medical News Today'
       ,
             snippet='We do the research so you can find trusted products for your health and
                  wellness. Read more about our vetting process. Humidifiers add moisture to
                  the air, which can benefit people with respiratory symptoms or dry skin.
                  There are several ways to use humidifiers in the home or office, but there
                  are also some risks.'),],
grounded_summarization='According to [link_id=1] and [link_id=3], humidifier is a device that
     adds moisture to the air to prevent dryness that can cause irritation in many parts of
     the body.'),
thoughts="Now that we know what humidifier is, we can look into the white powdery residual.",
),
Search(query='white powdery residual in humidifier',
thoughts="Let's now look into the white powdery residual.",
),
SelectLink(selected_links=[
  ResultItem(link_id=10, link_text='What is white dust and should I be concerned about it? -
       HVAC.com',
             snippet='White dust is usually caused by mineral content in the water that goes
                  into a humidifier. When the mist lands on furniture or other surfaces and
                  dries, it can sometimes leave behind dried mineral deposits, or "white dust
                  ."'),
             link_text='What is white dust and should I be concerned about it? - HVAC.com',
                  snippet='White dust is usually caused by mineral content in the water that
                  goes into a humidifier. When the mist lands on furniture or other surfaces
                  and dries, it can sometimes leave behind dried mineral deposits, or "white
                  dust."')
  ResultItem(link_id=11, link_text='Why is White Dust Coming Out of My Humidifier?',
             snippet='The white dust that you are finding on surfaces around your home is
                  likely dried-up minerals from water vapor created by the humidifier. These
                  minerals are natural in our home water supply. If you&#39;ve ever heard of
                  the term "hard water," it&#39;s referring to water with high levels of
                  calcium and magnesium.'),],
grounded_summarization='According to [link_id=10], and [link_id=11], the white powdery
     residual is caused by the mineral content in the water.'),
thoughts="We have all the information to answer the question.",
),
Terminate(thoughts="We have all the information to answer the question."),
]
ORIGINAL_QUESTION: str = "What's the white powdery residual often found on a humidifier?"

ACTION_SELECTED: Answer = Answer(thoughts="The first search ('what is humidifier') is probably
      not what the questioner cares, so let's omit that in the answer. We can focus more on
     the question itself.", answer="The white powdery residue in the humidifier is usually
     caused by mineral content in the water, according to [link_id=10]. For example, water may
      contain high levels of calcium and magnesium [link_id=11].")  # [END]

##########################
# Example 2:
##########################
```

Listing 4: *Self-check prompt (prefix)*

```
"""Check whether the ANSWER addresses the ORIGINAL_QUESTION and whether the ANSWER is based on
      SelectLinks in PAST_ACTIONS."""
from dataclasses import dataclass
from typing import List, Tuple


class Action:
  """Base class for different actions."""

class Check_Answer(Action):
  """Check whether the ANSWER addresses the ORIGINAL_QUESTION."""

  def __init__(self, passed: bool) -> None:
    self.passed = passed

class Revise_Answer(Action):
  """Revise the answer if it did not pass the check, based on information from SelectLinks in
       PAST_ACTIONS."""

  def __init__(self, revised_answer: str) -> None:
    self.revised_answer = revised_answer

  ...


@dataclasses.dataclass
class ResultItem:
  """Single search result, with link id, link title and snippet.

  Attributes:
    link_id: A unique integer id of this link.
    link_text: The title of the link.
    snippet: The snippet from the page that's relevant to the query
  """

  link_id: int
  link_text: str
  snippet: str
```

Listing 5: *Relevance self-check (fragment)*

```
##########################
# Example 1:
##########################

ORIGINAL_QUESTION: str = 'How to exclude a website from Google search'
PAST_ACTIONS: List[Action] = [
Search(query='exclude site in google search',
thoughts="We simply the ORIGINAL_QUESTION to a good search query.",
),
SelectLink(selected_links=[
  ResultItem(link_id=2, link_text='How to Exclude a Website from Google Search Results -
       Keyword',
             snippet="You can exclude a domain or even a specific page from appearing on
                  Google search results. Here's how ..."),
  ResultItem(link_id=3, link_text='How do I exclude/block specific web sites from search
       results?'),
             snippet="In the absence of a browser extension, the simplest way is to add -site:
                  website_name after your search terms. ... I can't get rid of books.google.
                  com results. I' ..."),
  ResultItem(link_id=6, link_text="Q&A: Can you exclude a specific website from Google's
       search results?"),
             snippet='Jul 31, 2022 . All you have to do is simply append -site:pinterest.com
                  to the end of your search query. For example, if you were searching for the
                  term "paper ...'),
  ],
grounded_summarization='[link_id=3] mentions that we can add -site:website_name to the google
     search query to exclude a website. [link_id=6] provides an example.'),
thoughts="We found enough info for a good answer to the ORIGINAL_QUESTION.",
),
Terminate(thoughts="This is a good answer to the question.")
Answer(thoughts="We summarize the relevant info found in PAST_ACTIONS.", answer="To exclude a
     website from Google search result, you can add -site:website_name after your search terms
      according to [link_id=3]. For example, if you want to exclude pinterest.com, you can
     simply add -site:pinterest.com according to [link_id=6].")
]
ORIGINAL_QUESTION: str = 'How to exclude a website from Google search'
ANSWER: str = 'To exclude a website from Google search result, you can add -site:website_name
     after your search terms according to [link_id=3]. For example, if you want to exclude
     pinterest.com, you can simply add -site:pinterest.com according to [link_id=6].'

# The ANSWER is directly addressing the ORIGINAL_QUESTION.
# The ANSWER refers to [link_id=3] which correctly supports the core statement of adding -site
     :website_name. The example from the answer is indeed from [link_id=6].
ACTION_SELECTED: Command = Check_Answer(passed=True)  # [END]

##########################
# Example 2:
##########################
```

Listing 6: *Grounding self-check (fragment)*

```
##########################
# Example 1:
##########################

ORIGINAL_QUESTION: str = "What did Jim Betts' competitor in the 1980 election achieve in 1962?
     "
PAST_ACTIONS: List[Action] = [
Search(query='who was Jim Betts competitor in 1980',
thoughts="This question is asking about the achievement of Jim Betts' competitor in 1962. Let'
     s first search for who this competitor is.",
),
SelectLink(selected_links=[
  ResultItem(link_id=1, link_text='Jim Betts (politician) - Wikipedia',
             snippet='Jim Betts is a former member of the Ohio House of Representatives. He
                  ran against incumbent U.S. Senator John Glenn in 1980. He lost and ran in
                  1982 for lieutenant governor.'),],
grounded_summarization="According to [link_id=1], Jim Betts' competitor in 1980 was John Glenn
     ."),
thoughts="Now that we know who the competitor is, we can search for his achievements in 1962."
     ,
),
Search(query='John Glenn achievements 1982',
thoughts="Now that we know who the competitor is, we can search for his achievements in 1982."
     ,
),
SelectLink(selected_links=[
],
grounded_summarization='Nothing is selected.'),
thoughts="The query 'John Glenn achievements 1982' is not very good. It should be 'John Glenn
     achievements 1962'.",
),
Search(query='John Glenn achievements 1962',
thoughts="John Glenn is the person we are looking for. Let's change the date to 1962.",
),
SelectLink(selected_links=[
  ResultItem(link_id=19, link_text='John Glenn | Biography, Accomplishments, & Facts |
       Britannica',
             snippet="Glenn, Jr., (born July 18, 1921, Cambridge, Ohio, U.S.-died December 8,
                  2016, Columbus, Ohio), the first U.S. astronaut to orbit Earth, completing
                  three orbits in 1962. (Soviet cosmonaut Yuri Gagarin, the first person in
                  space, had made a single orbit of Earth in 1961.) Glenn joined the U.S.
                  Naval Reserve in 1942."),],
grounded_summarization="According to [link_id=19], John Glenn's achievement in 1962 was to
     become the first American to orbit the Earth."),
thoughts="We have all the information to answer the question.",
),
Terminate(thoughts="We have all the information we need to answer the ORIGINAL_QUESTION. Let's
      stop searching and answer the question.")
Answer(thoughts="We have the answer to the question.", answer="Jim Betts\' competitor in the
     1980 election was John Glenn. And John Glenn\'s achievement in 1962 was to become the
     first American to orbit the Earth.")
]
ORIGINAL_QUESTION: str = "What did Jim Betts' competitor in the 1980 election achieve in 1962?
     "
ANSWER: str = 'Jim Betts\' competitor in the 1980 election was John Glenn [link_id=1]. And
     John Glenn\'s achievement in 1962 was to become the first American to orbit the Earth [
     link_id=19].'

# ANSWER directly addresses the ORIGINAL_QUESTION.
# [link_id=1] indeed says that Jim Betts\' run against John Glenn in 1980, consistent with
     ANSWER.
# [link_id=19] indeed says John Glenn is the first US astronaut to orbit the earth, consistent
      with ANSWER.
ACTION_SELECTED: Command = Check_Answer(passed=True)  # [END]

##########################
# Example 2:
##########################
```

Listing 7: *Auto-eval (full prompt)*

```
"""Check whether the ANSWER implies the REF_ANSWER to the question."""

def Check_Answer(ORIGINAL_QUESTION, ANSWER, REF_ANSWER):
  """Check whether the ANSWER implies the REF_ANSWER."""
  # todo

##########################
# Example 1:
##########################

ORIGINAL_QUESTION: str = 'Are McIntyre Mines and Copperfields Mine in the same country?'
ANSWER: str = 'According to [link_id=1], Copperfields Mine is in Ontario, Canada. According to
      [link_id=4], McIntyre Mines is also in Ontario, Canada. So yes, they are in the same
     country.'
REF_ANSWER: str = 'yes'

# the ANSWER implies the answer to the original question is yes, this is consistent with the
     REF_ANSWER.
Check_Answer(ORIGINAL_QUESTION, ANSWER, REF_ANSWER) = True  # [END]

##########################
# Example 2:
##########################

ORIGINAL_QUESTION: str = 'Which Director, James Richard Kelly or Paul L. Stein has a list of
     67 films to his credit? '
ANSWER: str = 'According to [link_id=1] and [link_id=2], James Richard Kelly has 5 films to
     his credit.'
REF_ANSWER: str = 'Paul Ludwig Stein (4 February 1892 - 2 May 1951) was an Austrian-born film
     director with 67 films to his credit.'

# the ANSWER does not imply the REF_ANSWER because ANSWER does not mention Paul Ludwig Stein.
Check_Answer(ORIGINAL_QUESTION, ANSWER, REF_ANSWER) = False  # [END]

##########################
# Example 3:
##########################

ORIGINAL_QUESTION: str = 'Are the genuses Michelia and Stenomesson in the same family?'
ANSWER: str = 'Based on the information we have, we are not sure whether the genuses Michelia
     and Stenomesson are in the same family [link_id=3] does not contain this information.'
REF_ANSWER: str = 'no'

# The ANSWER did not determine whether or not Michelia and Stenomesson are in the same family.
# The REF_ANSWER implies that they are not in the same family.
# Thus we cannot infer the REF_ANSWER given the ANSWER.
Check_Answer(ORIGINAL_QUESTION, ANSWER, REF_ANSWER) = False  # [END]

##########################
# Example 4:
##########################

ORIGINAL_QUESTION: str = 'In what year was the winner of the 44th edition of the Miss World
     competition born?'
ANSWER: str = 'According to [link_id=2], the winner of the 44th edition of the Miss World
     competition is Aishwarya Rai.\nAccording to [link_id=4], she was born in 1973.'
REF_ANSWER: str = '1973'

# The ANSWER implies 1973 as the final answer to ORIGINAL_QUESTION.
# REF_ANSWER implies 1973 as the final answer to ORIGINAL_QUESTION, too.
Check_Answer(ORIGINAL_QUESTION, ANSWER, REF_ANSWER) = True  # [END]

##########################
# Example 5:
##########################

ORIGINAL_QUESTION: str = 'Skull & Bones is a mini-series by a man that is an editor of what
     two comics?'
ANSWER: str = 'The question is ill-formed or out-of-date. We should ask for clarification from
      the asker.'
REF_ANSWER: str = 'Marvel Comics and DC Comics'

# The ANSWER did not provide any answer to ORIGINAL_QUESTION.
Check_Answer(ORIGINAL_QUESTION, ANSWER, REF_ANSWER) = False  # [END]

##########################
# Example 6:
##########################
```

Listing 8: *Reward model (full prompt)*

```
"""Rater Instructions:
- The goal of this rating is to filter out bad actions, so that they'll be excluded from the
     fine-tuning dataset.
- Overall, we want to the agent to produce relevant and grounded answers with minimal steps.
     Anything that deviates from this goal is considered bad.
- If any element (thoughts, comments etc.) is empty, then it's automatically bad.

##########################################



*** Model Can See:
```
{inputs}
```

*** Model Output #1:
```
{action1}
```

*** Model Output #2:
```
{action2}
```

*** Model Output #3:
```
{action3}
```

*** Model Output #4:
```
{action4}
```

##########################################

Your Instructions:
- Choose the best model output based on the rater's instructions.
- Don't assume in your decision that the model knows anything outside of "Model Can See"
     section.
- Be specific in your explanation.

Output 3 lines when answering and make sure to follow the precise format.

Explanation: why you think model output #X is the best
Answer: #X
Ranking: #X > #Y > ...
"""
```